SMART CITIES · WATER DOMAIN

Smart Water Management for Urban Green Infrastructure

A low-cost, network-resilient edge-IoT irrigation node with deep-learning soil analysis

Project Report & Demonstration · Design of Smart Cities

Cities irrigate a great deal of land — on a timer

Parks, urban forests, road medians, green belts, green roofs and community gardens

A timer cannot sense

It does not know it rained yesterday, or that one bed drains faster than another.

Both failures at once

The same green space is over-watered in places and under-watered in others.

Nobody notices

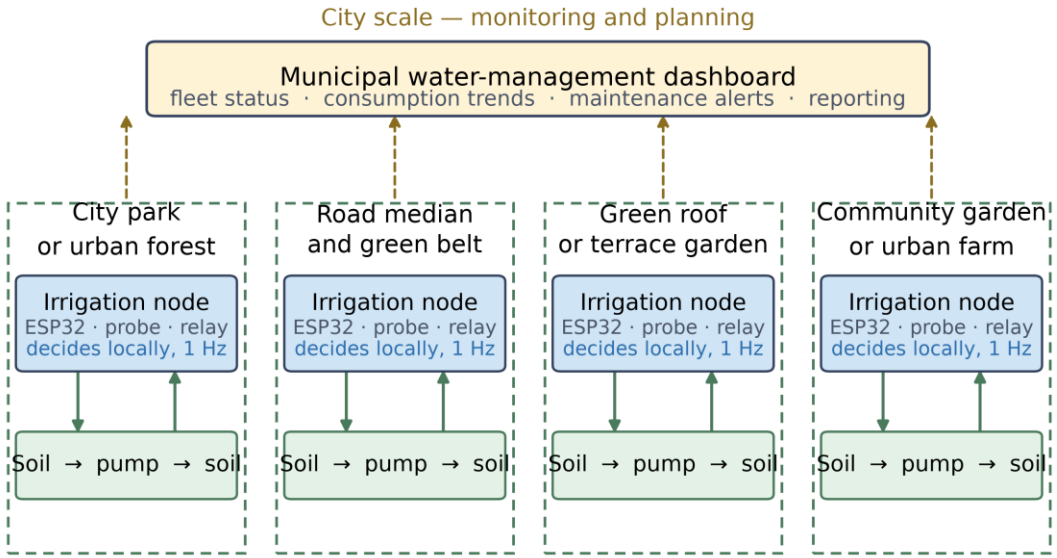
Open-loop systems report success whether or not water actually reached the soil.

Green infrastructure is growing in importance — vegetation moderates the urban heat island, manages stormwater and improves air quality. Cities are asked to maintain more of it, more reliably, without using more water.

Demand-driven irrigation is the direct response to that pressure.

What we built

One node per irrigation zone, replicated across heterogeneous urban green space



Solid loop: the irrigation decision is taken on the node and never leaves it. Dashed: telemetry upward only — the city layer reports, it cannot actuate.

If the city network fails, every node keeps watering.

₹970
cost per node

1 Hz
local decision rate

0
network dependency

SECTION ONE

SRS — Requirements

Software Requirements Specification — what the system must do, and what it must survive

SRS — Requirements

Ten functional and six non-functional requirements; the non-functional ones decide whether it scales

Functional

- Measure soil moisture, air temperature and humidity
- Switch the pump automatically from the measurement
- Use two thresholds with a dead band, not one set point
- Emit a telemetry record at a fixed interval
- Indicate state locally, visible without instruments
- Classify soil type and condition from a photograph
- Produce weather-aware watering advice
- Provide monitoring, reporting and manual override

Non-functional

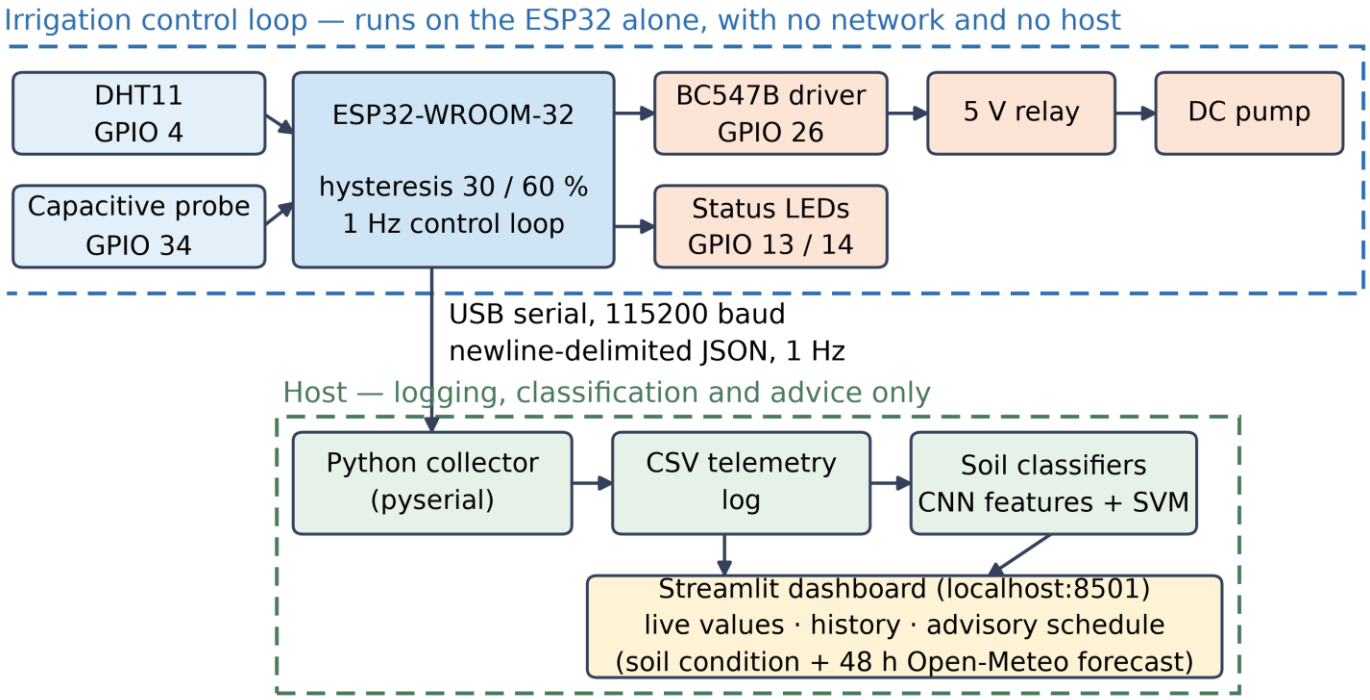
- The irrigation decision is taken on the node, never remotely
- Unit cost low enough to deploy in quantity
- Failure of a secondary sensor must not stop irrigation
- The monitoring layer must be incapable of actuation
- Commodity parts, for municipal procurement and repair
- Undetectable failure modes must be identified and documented

SECTION TWO

SDS — Design

Software Design Specification — architecture, hardware, data and the control law

Four layers, one boundary that matters



Everything needed to keep vegetation watered lives on the microcontroller. Every layer above it is additive: logging, classification, advice, reporting.

CENTRAL DESIGN DECISION

The node never asks a server whether to irrigate

On a farm that is convenience. Across a fleet of four hundred nodes it is the difference between a network outage degrading visibility and a network outage stopping service.

Network fails

Every node keeps watering

Host fails

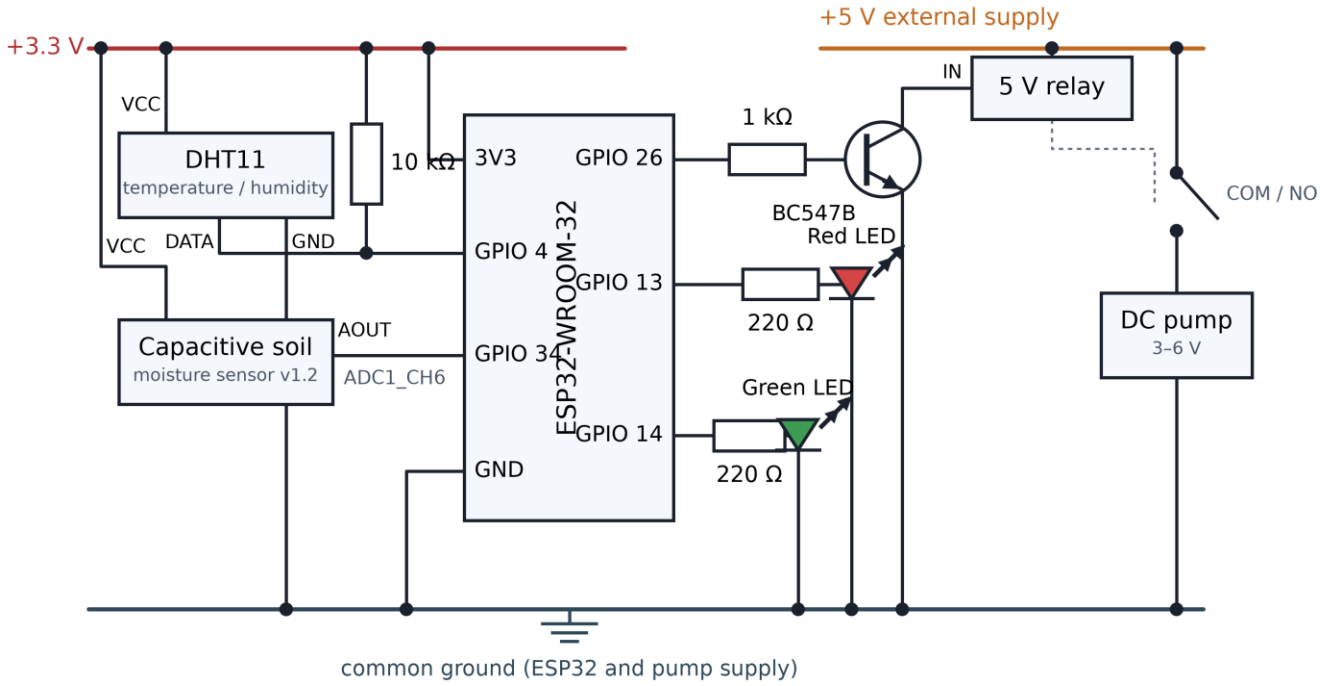
Every node keeps watering

Node fails

One zone affected, not the fleet

Hardware and cost

Commodity parts only — unit cost is a design parameter, not an outcome



Bill of materials

ESP32-WROOM-32	₹284
Capacitive probe v1.2	₹83
DHT11 module	₹61
5 V relay module	₹42
Pump, BC547B, LEDs, resistors, wiring	₹500
Total per node	**₹970**

100 zones ≈ ₹97,000 **400 zones ≈ ₹3.9 lakh**

Hardware only — excludes enclosures, plumbing, installation and maintenance. Given to show the order of magnitude at which this becomes a procurement question.

SECTION THREE

Implementation

Firmware, host software and the assembled node

The control loop, once per second

Self-contained: no host, no network, no cloud service

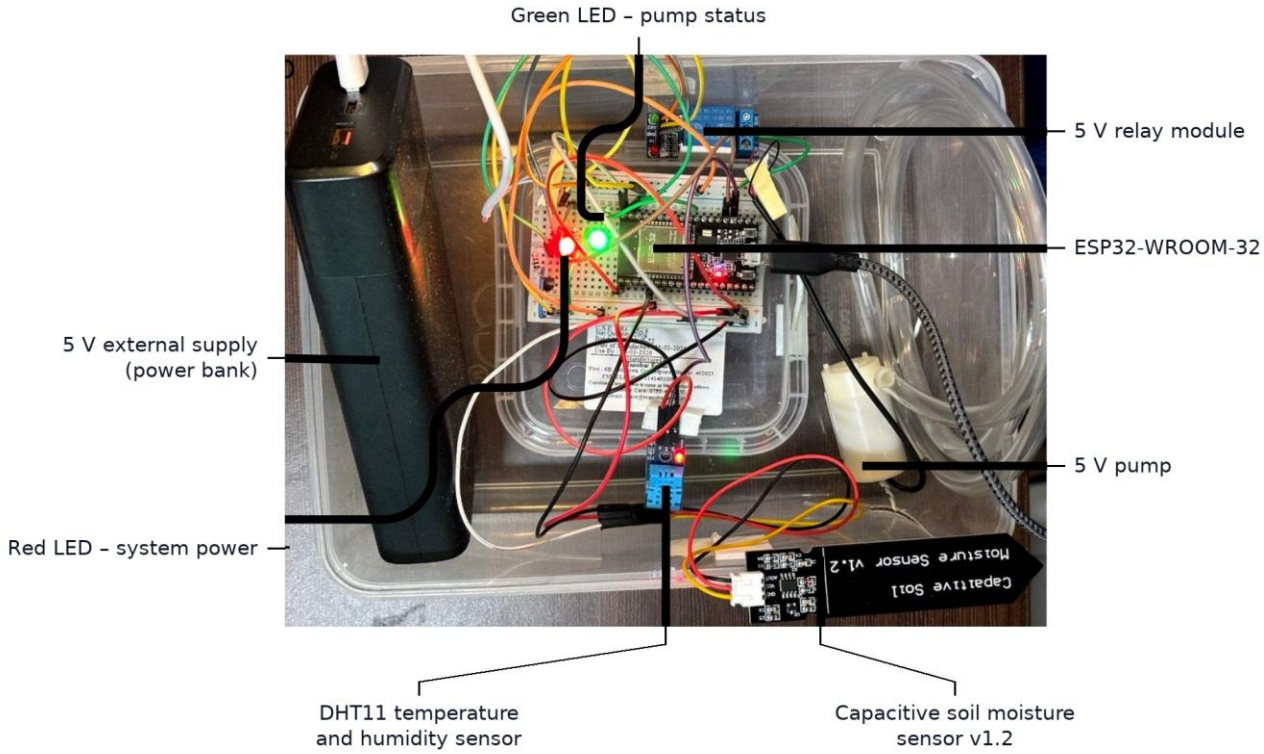
1. **Read sensors** — DHT11 on GPIO 4, capacitive probe on GPIO 34 as a raw 12-bit value
2. **Convert and clamp** — Two-point calibration to 0–100 %; the clamp also guards a disconnected probe
3. **Apply hysteresis** — Pump on below 30 %, off above 60 %, hold state in between
4. **Update indicators** — Red steady for alive; green blinks at 250 ms while the pump runs
5. **Emit telemetry** — One JSON record at 115200 baud, then wait one second

Stack

- C/C++ on the Arduino core
- USB serial, 115200 baud, JSON
- Python 3.11 + pyserial
- TensorFlow / Keras
- scikit-learn
- Streamlit + Plotly
- ReportLab
- Open-Meteo forecast API

The assembled node

Probe in a sealed soil container that can be wetted and left to dry — repeatable control of the variable under test



SECTION FOUR

Testing

Not only whether it works, but what it cannot see

Seven fault-injection tests

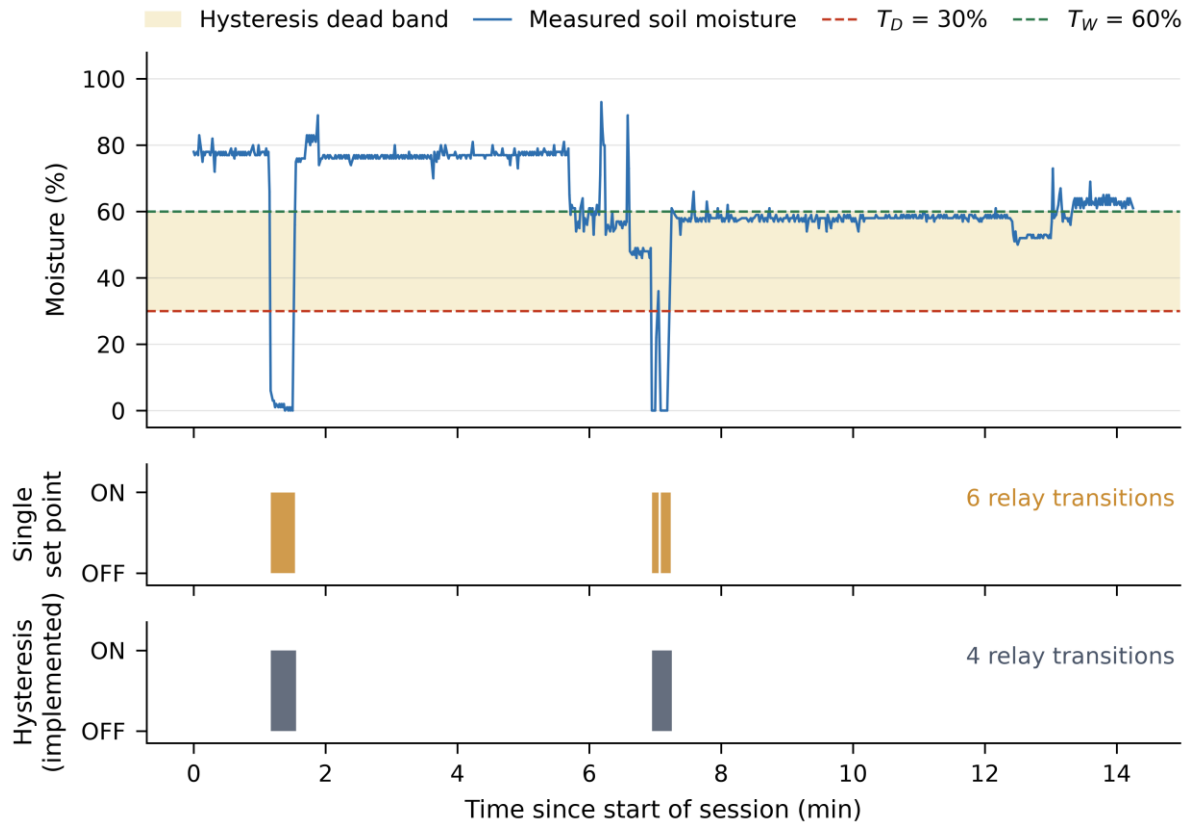
Five behaved as specified; two did not, and both are reported

Test	Condition applied	Result
TC-01	Moisture 65 %, normal conditions	Pump off, telemetry sent — pass
TC-02	Moisture 22 %	Pump on, green LED blinking — pass
TC-03	Moisture 45 % with pump already on	State held, no chatter — pass
TC-04	Moisture exactly 30 %	Pump off, matches the rule — pass
TC-05	DHT11 data line removed	Nulls sent, irrigation continued — pass
TC-06	Moisture probe unplugged	Fail-safe triggered, but fault not detected
TC-07	Pump supply removed while running	**Pump stopped, firmware kept reporting it on**

TC-07 is the most consequential result. The relay is commanded but never measured, so a dead pump is reported as running. In a municipal deployment this persists until vegetation visibly declines — and every record in between says the site was watered.

Control-law validation

8,171 telemetry records across 33 logging sessions, median interval 1.00 s



17 %

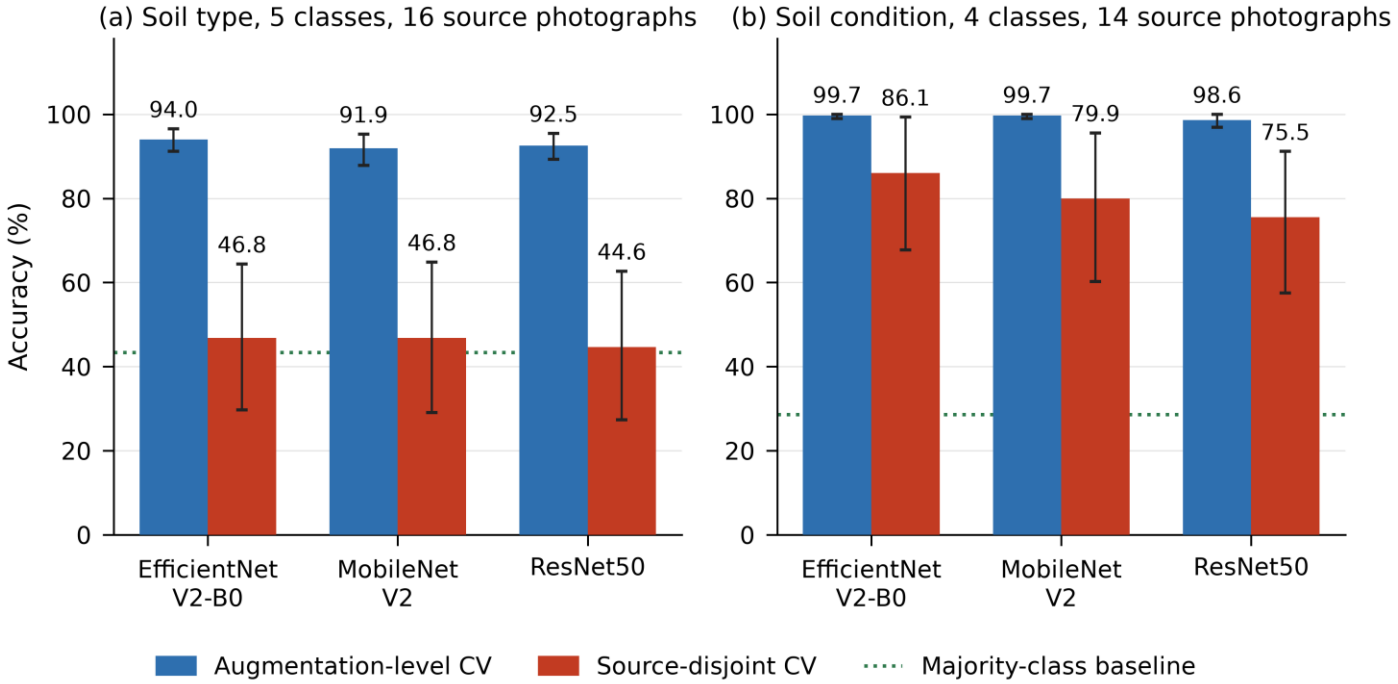
fewer relay transitions
than a single set point

6.6 %

of readings fall inside
the 30–60 % dead band

How the soil classifier was evaluated

The same classifier and features, tested two ways



Two protocols

Augmentation-level
Shuffle all expanded samples, then split. Near-identical crops of one photograph land on both sides.

Source-disjoint
Hold out whole photographs. The classifier is tested only on soil it has never seen.

Soil type falls from 91.9–94.0 % to 44.6–46.8 % — close to the 43.3 % obtainable by always guessing the most common class. Most of the apparent accuracy came from recognising photographs, not soils. Soil condition holds up at 75.5–86.1 % and is the usable one.

SECTION FIVE

Outcome

What was achieved, what was not, and what it means at city scale

Measured outcomes

15 of 16 requirements fully met; one partially met

₹970 per node	**17 %** fewer actuations	**5 / 7** fault tests passed
8,171 records logged	**86.1 %** soil condition, unseen	**46.8 %** soil type, unseen

The infrastructure objective was met. The analytics objective is qualified: soil condition is usable, soil type is not — and would have looked highly accurate under conventional evaluation.

What this means at city scale

Visibility without fragility

Because the monitoring layer has no actuation path, adding city-wide visibility does not add a city-wide failure mode. An operations centre can lose four hundred nodes from view without any of them stopping work.

Cost decides coverage

At ₹970 a node, the limiting factor on how much green space a city can instrument stops being hardware budget and becomes installation and maintenance.

Silent failure is a governance problem

A dashboard showing green for a dead pump is worse than no dashboard. Undetectable failure modes must be found before deployment, not after.

Accuracy claims need protocols

Smart-city systems increasingly justify decisions by citing model accuracy. A figure quoted without stating how the data were split cannot support a procurement decision.

Limitations and next steps

Limitations

- A pump that loses power is reported as running
- A drifted or fouled probe is trusted
- Water saving not measured — no flow meter fitted
- Soil type not deployable at its measured accuracy
- Validated in a container, not a live municipal site
- One probe represents a whole irrigation zone
- Telemetry link is wired USB serial

Next steps

- ACS712 current sensor — ₹145, closes the TC-07 gap
- Pulse-response probe check to catch a fouled probe
- Inline flow meter, so saving is reported in litres
- Wi-Fi, LoRaWAN or NB-IoT backhaul for real city scale
- Multi-zone valve control from one node
- Several hundred field photographs for the classifier
- Integration with municipal asset-management workflow

IN CLOSING

A controller that reports irrigation it is not performing, and a classifier that reports accuracy it does not have, fail in the same way

Both look correct from the outside. Neither was discovered by watching the system work — both were found only by testing specifically for them. For infrastructure a city intends to trust without supervision, that testing is not an optional extra.

Thank you